

Chapter 1

Introduction

1.1 Overview / Background

Credit card fraud has become one of the most common and damaging financial crimes in the modern digital era. With the rapid growth of e-commerce, online transactions, and digital banking systems, fraudulent activities have increased significantly. Fraudsters frequently employ sophisticated techniques such as identity theft, card skimming, phishing, and unauthorized online purchases to exploit vulnerabilities in financial systems. As a result, banks and financial institutions face large economic losses and reduced customer trust. Detecting such fraud manually is extremely challenging due to the massive volume, high speed, and complex nature of transaction data. Traditional rule-based fraud detection systems are not capable of adapting to new fraud patterns. Machine Learning (ML) techniques, however, offer an effective and automated approach to analyze large datasets, identify hidden patterns, and classify transactions as legitimate or fraudulent with high accuracy.

Machine learning algorithms such as Support Vector Machines (SVM), Random Forest, and XGBoost help in processing imbalanced datasets, performing feature extraction, and learning dynamic fraud patterns. These intelligent models can continuously adapt, improve accuracy, reduce false positives, and enhance the security of digital financial systems. Hence, machine learning-based fraud detection has become an essential component in modern financial technologies.

1.2 Problem Statement

Credit card fraud detection is increasingly challenging due to the rapid rise in digital payments, mobile banking, and e-commerce transactions. As transaction volumes grow, fraudulent activities also evolve, becoming more sophisticated and difficult to identify using traditional rule-based systems. These older systems rely on predefined patterns, which makes them ineffective when new or unseen types of fraud emerge. One of the major challenges in detecting fraud is the highly imbalanced nature of real-world transaction datasets, where legitimate transactions vastly outnumber fraudulent ones. This imbalance often causes models to misclassify fraud cases, leading to high false-negative rates. Additionally, many existing methods struggle with delayed detection, resulting in financial loss before an alert is raised.

Another concern is the high rate of false alarms, which increases manual verification workload and affects user experience.

To overcome these issues, there is a need for an intelligent, adaptive, and data-driven approach. A machine learning–based fraud detection system can analyze multiple features such as transaction amount, frequency, location patterns, and user behavioral trends. By learning from historical data and identifying subtle anomalies, such a system can significantly improve accuracy, enhance early detection, and reduce false positives, providing a more secure and efficient fraud prevention mechanism.

1.3 Objectives

The objectives of the project are:

- To design an automated system capable of detecting fraudulent credit card transactions.
- To apply machine learning algorithms for accurate and efficient fraud classification.
- To preprocess and analyze large transactional datasets for extracting meaningful features.
- To minimize false positives by ensuring no genuine transaction is incorrectly marked as fraud.
- To improve fraud detection performance using classification techniques such as SVM, Random Forest, and XGBoost.
- To provide an easy-to-use interface for analyzing, training, and testing the fraud detection model.

1.4 Scope of the Project

This project focuses on building a machine-learning-based credit card fraud detection system using supervised learning methods. The primary objective is to develop a reliable model capable of classifying transactions as legitimate or fraudulent by learning patterns from historical data. The workflow includes several key stages such as dataset preprocessing, handling missing values, feature extraction, normalization, model training, performance evaluation, and final classification. During preprocessing, the dataset is cleaned and transformed to ensure that the model receives high-quality input. Feature extraction helps identify meaningful attributes such as transaction amount, frequency, and behavioral indicators

that play a critical role in detecting anomalies. Various supervised learning algorithms are trained and tested to identify the one that provides the best overall performance.

1.5 Methodology Overview

The proposed system follows a simple, structured methodology:

1. **Dataset Collection:** The project begins by obtaining a publicly available credit card transaction dataset containing both legitimate and fraudulent records.
2. **Data Preprocessing:** The raw data is cleaned by removing duplicates, handling missing values, filtering noise, and applying feature scaling to ensure consistent model input.
3. **Feature Extraction:** Important features related to transaction amount, frequency, and behavior patterns are extracted to enhance the model's ability to detect anomalies.
4. **Model Training:** Multiple supervised learning algorithms—including SVM, Random Forest, Decision Tree, and XGBoost—are trained using the processed dataset.
5. **Model Evaluation:** Each model is assessed using performance metrics such as accuracy, precision, recall, and F1-score to determine effectiveness.
6. **Model Selection & Classification:** The best-performing algorithm is selected and used to classify new or test transactions as fraudulent or legitimate.
7. **System Implementation:** The system integrates modules for data analysis, model building, and model testing, all accessible through a user-friendly interface.

1.6 Project Significance

This project plays a crucial role in strengthening the security and reliability of digital financial transactions. With the rapid growth of online banking, mobile payments, and e-commerce platforms, detecting fraudulent activities quickly and accurately has become essential. Traditional rule-based systems often fail to identify evolving fraud patterns, leading to financial

losses and operational inefficiencies. By incorporating machine learning techniques, this project provides an intelligent and automated approach to identifying suspicious transactions at an early stage.

The system analyzes multiple transaction features and learns behavioral patterns, enabling it to detect even subtle anomalies that may indicate fraud. This significantly reduces the chances of undetected fraudulent activities and minimizes false positives, allowing financial institutions to make faster and more informed decisions. Automated classification also reduces manual verification time, improving overall operational efficiency.

Chapter 2

Literature Review

2.1 Reference 1

A recent study on fraudulent credit card detection using Support Vector Machines (SVM) and Decision Trees emphasizes the rapid increase in electronic transactions and the corresponding rise in financial fraud risks. The authors proposed a hybrid machine learning approach that combines the strengths of both SVM and Decision Trees to enhance overall classification accuracy. Their results demonstrated that the hybrid model effectively reduces false positives and improves the reliability of fraud detection compared to traditional standalone techniques. However, the study also highlights a significant limitation: the model struggles to perform well when applied to large and highly imbalanced datasets, which are common in real-world financial environments where fraudulent transactions make up only a small percentage of the total data. This imbalance often leads to poor recall and a higher number of undetected fraud cases.

The proposed system in our project addresses these limitations by incorporating robust preprocessing and feature extraction methods before model training. Techniques such as noise removal, feature scaling, normalization, and selection of meaningful transaction attributes help the model handle imbalance more effectively. By preparing cleaner and more representative data, our system improves classification performance and enhances the ability to detect fraudulent transactions accurately, even within large datasets.

2.2 Reference 2

A machine-learning-based approach for detecting fraud in mobile payment systems highlights the growing need for intelligent techniques capable of analyzing massive volumes of transactional data. As mobile payments continue to rise, so does the risk of fraudulent activities that traditional rule-based systems often fail to detect. The study emphasizes the use of both supervised and unsupervised learning methods to identify suspicious patterns, anomalies, and irregular behavior in real time. The authors employed feature selection techniques to identify the most relevant transaction attributes and applied sampling methods to improve model balance, which led to notable improvements in classification accuracy and efficiency.

Despite these advantages, the model presented in the study had a key limitation: it lacked thorough evaluation across diverse datasets with varying levels of imbalance, transaction types, and user behavior patterns. Without extensive validation, the generalizability and robustness of the model remain uncertain, especially in real-world financial environments.

Our proposed system addresses this limitation by integrating multiple machine learning algorithms—including SVM, Random Forest, Decision Tree, and XGBoost—and performing a comparative analysis of their performance. By evaluating models on a structured, preprocessed dataset and comparing accuracy, precision, recall, and F1-score, our system provides a more reliable and comprehensive solution for fraud detection.

2.3 Reference 3

Another research study introduced a hybrid fraud detection framework that integrates Hidden Markov Models (HMM), behavior-based analysis, and genetic algorithms to identify unusual transaction patterns. HMM was used to model sequential user behavior, while genetic algorithms optimized feature sets and detection thresholds. This combination improved detection accuracy by capturing both temporal patterns and behavioral deviations that traditional models often overlook. The system demonstrated strong performance in identifying sophisticated fraud attempts, especially those involving subtle or disguised behavioral changes. However, the study also revealed significant limitations. The hybrid model required extensive computational resources due to the complexity of HMM state transitions and the iterative nature of genetic algorithm optimization. As a result, training the model on large datasets became time-consuming and impractical for real-time or near-real-time applications. Moreover, the heavy computational workload limited the model's scalability across diverse financial environments.

Our proposed system addresses these challenges by using more efficient machine learning algorithms such as Support Vector Machines (SVM) and XGBoost. These algorithms provide faster training times, lower computational cost, and improved scalability while maintaining high detection accuracy. By combining efficient preprocessing, feature extraction, and algorithm selection, our approach delivers a more practical and resource-efficient solution suitable for large datasets and real-world fraud detection scenarios.

2.4 Reference 4

A study comparing the performance of Naive Bayes, K-Nearest Neighbors (KNN), and Logistic Regression for credit card fraud detection presented insightful results regarding model behavior on transactional data. The authors reported that KNN achieved the highest accuracy among the three algorithms, mainly due to its ability to classify data points based on similarity measures. This made KNN effective in identifying certain types of fraudulent patterns when the dataset was moderately balanced and feature distributions were well-defined. Naive Bayes and Logistic Regression, while fast and easy to implement, showed weaker performance because of their assumptions about data independence and linear decision boundaries.

Despite the advantages of KNN, the study faced significant limitations when dealing with highly skewed datasets, which is a common challenge in real-world financial environments where fraudulent transactions represent only a tiny portion of all records. The model's performance dropped notably due to imbalanced class distribution, and the lack of advanced feature engineering further constrained its ability to identify subtle fraud patterns. Limited preprocessing also affected the overall quality of model training.

Our proposed system addresses these issues by incorporating robust preprocessing techniques such as noise removal, normalization, and feature extraction. Additionally, advanced algorithms like XGBoost enhance classification performance by handling imbalance effectively and capturing complex fraud patterns with higher accuracy.

2.5 Reference 5

Support Vector Machine (SVM) classifiers have long been recognized as powerful tools in various pattern recognition and classification tasks due to their ability to handle both linear and non-linear data. In the referenced study, the authors utilized kernel functions—such as polynomial, radial basis function (RBF), and sigmoid kernels—to transform input data into higher-dimensional spaces. This enabled the SVM model to draw optimal separating boundaries between legitimate and fraudulent transactions, even when the data exhibited complex and non-linear relationships. The study demonstrated that SVM offers strong generalization capability and high detection accuracy when trained on well-structured datasets. However, the authors also noted a crucial limitation: the performance of SVM is highly dependent on the quality and relevance of input features. Without proper feature engineering and selection, the classifier may fail to capture subtle fraud patterns or may require extensive tuning to achieve optimal results. This makes SVM sensitive to noise, redundant information, and high-dimensional data, which can reduce detection effectiveness.

Our proposed system addresses this issue by incorporating a dedicated feature extraction module that processes and refines the dataset before model training. By selecting the most meaningful attributes and removing irrelevant variations, the system significantly enhances SVM performance and improves overall fraud classification accuracy.

2.6. LITERATURE SURVEY SUMMARY TABLE:

Sl. No	Paper Title	Publication Details	Problem Addressed	Author's Approach	Shortfalls & Proposed Solution
1	Fraud Detection using SVM & Decision Tree	Journal Paper	Credit card fraud in online transactions.	Hybrid ML model.	Struggles with imbalanced data; propose preprocessing + feature engineering.
2	Mobile Payment Fraud Detection	IEEE	Unauthorized mobile transactions..	Sampling + Feature selection.	Limited datasets; propose multiple models + comparative analysis..
3	Novel Approach for Credit Card Fraud Detection	Springer	Hybrid HMM-based fraud detection. .	HMM + GA .	High computation; propose efficient classifiers (SVM, XGBoost).

4	ML Techniques for Fraud Detection	Elsevier	Accuracy issues in classification. .	NB, KNN, LR.	Poor performance on imbalance; propose SVM + XGBoost with preprocessing.
5	SVM for Pattern Recognition	IEEE	Non-linear classification.	Kernel-based SVM.	Feature dependency; propose feature extraction module..

Chapter 3

Overview Of Proposed Methodology

3.1 Overview of the Proposed System

The proposed system is developed to accurately detect fraudulent credit card transactions using advanced machine learning classification algorithms. Unlike traditional rule-based fraud detection methods, which rely on fixed patterns and often fail to identify emerging fraud behaviors, this system adopts a data-driven and adaptive approach. It begins by collecting and analyzing historical transaction records, which serve as the foundation for training the predictive model. Through systematic preprocessing, the system performs essential tasks such as data cleaning, removal of duplicate entries, handling of missing values, elimination of noise, and normalization of numerical features. These steps ensure that the dataset is consistent, balanced, and suitable for reliable model training.

A dedicated feature extraction module then identifies the most relevant attributes—such as transaction amount, frequency, location behavior, and temporal patterns—to enhance classification accuracy. Multiple machine learning algorithms, including SVM, Random Forest, Decision Tree, and XGBoost, are trained on this refined dataset. Each model is evaluated based on performance metrics, and the best-performing classifier is selected to make predictions on new, unseen transactions.

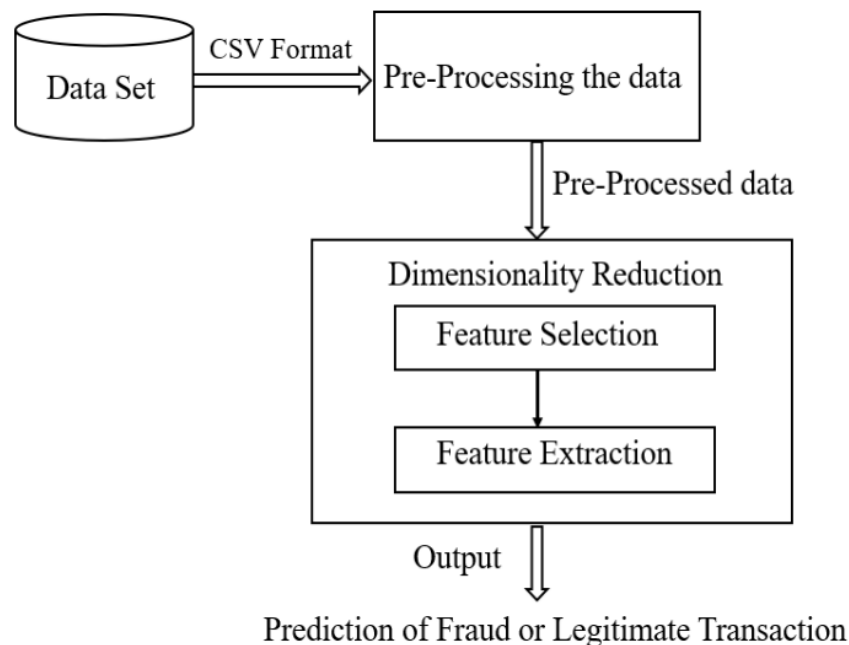
By combining preprocessing, feature engineering, and comparative model evaluation, the system achieves higher accuracy, reduced false positives, and improved reliability. This makes the proposed solution robust, efficient, and suitable for practical fraud detection applications.

3.2 System Architecture

System Architecture – 5 Key Components

1. **Preprocessing & Feature Engineering:** The system loads the transaction dataset, removes duplicates, handles missing values, eliminates noise, and extracts key features such as transaction amount, frequency, and behavioral patterns.

2. **Model Training Module:** Machine learning algorithms including SVM, Random Forest, Decision Tree, and XGBoost are trained on the preprocessed dataset to learn fraud patterns effectively.
3. **Model Evaluation Module:** All trained models are compared using performance metrics such as accuracy, precision, recall, and F1-score to identify the best-performing classifier.
4. **Prediction Module:** The selected model is used to classify new or unseen transactions as legitimate or fraudulent based on learned patterns.
5. **User Interface Module:** A simple and interactive GUI enables users to perform data analysis, train models, and test transactions for fraud detection.



3.3 Modules Description

1. Data Collection Module

Responsible for loading the credit card transaction dataset into the system. The dataset is stored in CSV format and contains transaction-related attributes.

2. Preprocessing Module

Handles duplicate removal, missing value treatment, outlier elimination, and data normalization. It improves dataset quality for accurate model training.

3. Feature Extraction Module

Extracts relevant features from the dataset such as transaction amount, frequency, CVV, billing address verification, time between transactions, and purchase patterns.

4. Classification Module

Uses ML algorithms (SVM, Random Forest, XGBoost, Decision Tree) to classify transactions. The algorithm with the best accuracy is selected for final model testing.

5. Model Testing Module

Allows users to input transaction details and predicts whether the transaction is fraudulent or legitimate.

6. GUI Module

Provides interactive screens for login, dataset upload, analysis, model building, and model testing.

3.4 Technologies and Tools Used

Technology/Tool	Purpose	Advantages
Python	Core programming language	Simple, extensive ML libraries
Flask	Web framework for UI	Lightweight and easy to integrate
Jupyter Notebook	Model-building environment	Interactive coding and visualization
HTML/CSS	Front-end development	Simple, customizable UI
Pandas/Numpy	Data preprocessing	Efficient data handling
Scikit-Learn	ML algorithms	Ready-made classification models
XGBoost Library	Boosted classifier	High accuracy and speed

3.5 Advantages of the Proposed System

1. Achieves higher accuracy compared to traditional rule-based fraud detection methods.
2. Effectively handles imbalanced datasets using preprocessing, normalization, and feature engineering.
3. Provides fast and automated fraud predictions, improving overall detection speed.
4. Reduces both false positives and false negatives, ensuring more reliable results.
5. Adapts to evolving fraud patterns through continuous learning from data.
6. Supports multiple ML algorithms such as SVM, Random Forest, Decision Tree, and XGBoost.
7. Selects the best-performing classifier using evaluation metrics for optimal detection performance.
8. Improves decision-making for financial institutions by delivering consistent and accurate predictions.

Chapter 4

Requirements Specification

4.1 Functional Requirements

The proposed fraud detection system includes several essential functional requirements to ensure accurate model training, efficient data processing, and reliable prediction results. These requirements guide the system's behavior, workflow, and interaction with the user. Each function plays a critical role in enabling fraud detection using machine learning techniques. The system must provide seamless data handling, model evaluation, and prediction capabilities. Below are the key functional requirements of the system.

1. Admin & Authentication

- Secure admin login access
- Credential verification
- Controlled access to system modules

2. Dataset Handling

- Upload transaction dataset (.csv)
- Validate dataset format
- Store dataset for further processing

3. Data Preprocessing

- Remove duplicates and noise
- Handle missing values
- Normalize and clean transaction attributes

4. Feature Engineering

- Extract key features for fraud detection
- Select relevant behavioral and transactional attributes
- Prepare optimized feature set for model training

5. Model Training & Evaluation

- Train ML models (SVM, RF, DT, XGBoost)

- Compare accuracy, precision, recall, F1-score
- Select the best-performing classifier

6. Prediction & Visualization

- Classify transactions as fraudulent or legitimate
- Display graphical attribute analysis
- Allow user input for real-time testing

7. System Interface

- Provides a simple and intuitive GUI
- Ensures smooth navigation across modules
- Displays results, graphs, and outputs clearly

8. Transaction Analysis

- Analyzes uploaded transaction patterns
- Detects unusual or suspicious behaviors
- Highlights high-risk transaction attributes

9. Model Management

- Saves trained models for reuse
- Loads previously stored models
- Maintains model version tracking

10. Performance Monitoring

- Monitors accuracy and detection trends
- Tracks false positives and false negatives
- Generates performance summaries for admin review

11. Reporting & Output

- Creates detailed fraud detection reports
- Exports results in CSV or text formats
- Summarizes insights for decision-making

4.2 Non-Functional Requirements

1. Performance

- System should process input data quickly
- Predictions must be generated without delay
- Response time should support smooth user interaction

2. Accuracy

- Fraud prediction must meet acceptable accuracy levels
- Model should minimize false positives and false negatives
- Accuracy must remain stable across different datasets

3. Usability

- Interface must be simple and user-friendly
- Navigation should be easy for both technical and non-technical users
- All features must be clearly labeled and accessible

4. Reliability

- System must produce consistent results during repeated tests
- Must operate without unexpected errors or failures
- Classification output should remain stable across sessions

5. Scalability

- Should efficiently handle large-sized datasets
- Must support increased transaction volumes in the future
- System performance should not degrade with data growth

6. Security

- Admin login must be secure and protected
- User credentials and data must remain confidential
- Sensitive transaction information must be encrypted

4.3 Software Requirements

1. Programming Language & Runtime

- Python 3.8+ is the project's primary language.
- Ensures compatibility with modern ML libraries and features.
- Use virtual environments (venv/conda) to manage dependencies.

2. Web Framework

- Flask for building the web-based interface and APIs.
- Lightweight and easy to integrate with ML model endpoints.
- Supports quick prototyping and deployment of prediction services.

3. Frontend Technologies

- HTML and CSS for creating the user interface.
- Simple layouts for dataset upload, model control, and result display.
- Can be extended later with JS frameworks if needed.

4. Development & Notebook Environment

- Jupyter Notebook for data exploration, preprocessing, and model experiments.
- VS Code as the primary code editor for development and debugging.
- Notebooks for reproducible analysis; VS Code for production-ready scripts.

5. Machine Learning Libraries

- Scikit-learn for classical ML algorithms and evaluation utilities.
- XGBoost for high-performance gradient boosting models.

6. Platform & OS

- Supports Windows 10 and Linux for development and deployment.
- Cross-platform compatibility using environment managers and Docker (optional).

4.4 Hardware Requirements

Hardware	Specification	Purpose
Processor	Intel Core i3 or above	For executing ML algorithms
RAM	5 GB or higher	Handles dataset preprocessing
Storage	500 GB HDD/SSD	Stores datasets and model files
System Speed	2.2 GHz	Smooth execution

4.5 Other Requirements

1. Dataset Requirement

- Kaggle Credit Card Fraud Detection dataset
- Contains labeled legitimate and fraudulent transactions
- Used for training, testing, and evaluating ML models

2. Internet Requirement

- Stable internet connection required for dataset download
- Ensures smooth access to Kaggle resources
- Supports installation of external libraries and dependencies

3. Package Management

- Python package manager (pip) for installing libraries
- Handles ML packages like Scikit-learn, XGBoost, Pandas, NumPy
- Ensures all dependencies are properly managed and updated

Chapter 5

Project Design

5.1. System Design

The system is designed to provide a seamless end-to-end workflow for device scanning, analysis, and risk classification.

It integrates modules for preprocessing, feature extraction, and classification to ensure accurate and efficient results.

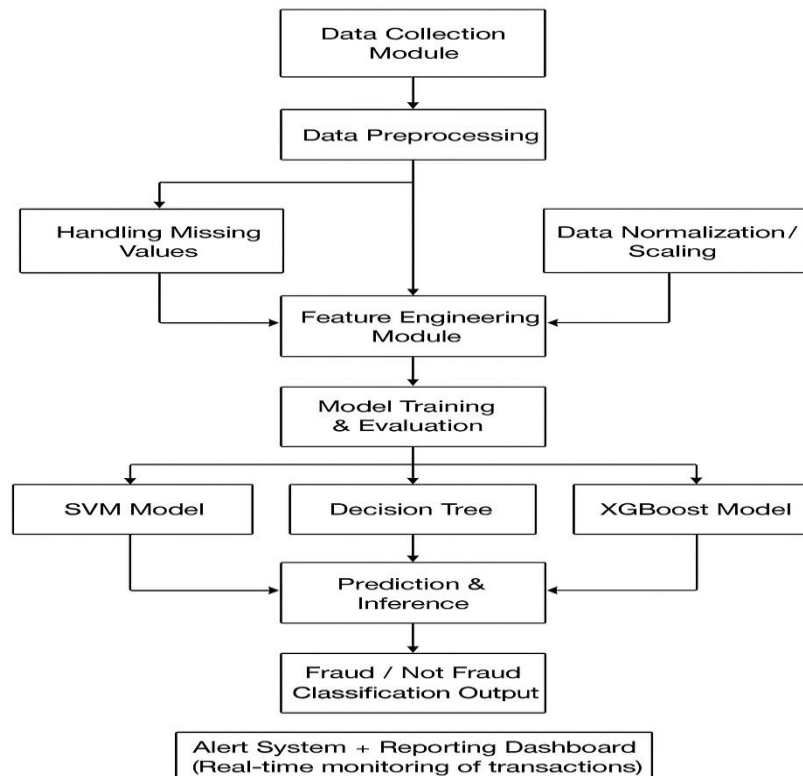
A centralized architecture enables smooth data flow between components while maintaining system reliability.

The design prioritizes performance, real-time processing, and user-friendly interaction.

Overall, the system architecture ensures fast scanning, precise detection, and clear visualization of outcomes.

5.1.1. User Interface Design

The user interface is developed to be simple, intuitive, and result-focused, ensuring users can perform scanning and view outcomes with minimal effort. The entire workflow is presented on a unified dashboard, offering centralized access to all essential functionalities. A clearly highlighted action button allows the user to start the scan instantly, triggering the complete process through a single API interaction. Throughout the scan, the interface delivers real-time progress updates, including device discovery and classification status, supported by low-latency WebSocket communication. Once the scan completes, the results are displayed in a dynamic and easily sortable table containing key details such as IP Address, MAC Address, Vendor, Open Ports, and the generated Risk Classification. To enhance readability, the classification outcomes are visually coded using intuitive colors—green indicating low risk, yellow/orange for medium risk, and red representing high-risk devices—allowing users to quickly assess network safety.



The user interface is built using a web-based dashboard (React/Streamlit) designed for analysts, banking staff, and fraud investigation teams.

Key points explaining the UI design:

1. Login / Authentication Page

- Provides secure login via username/password or enterprise SSO.
- Ensures only authorized bank officers or analysts can access transaction data.
- Redirects users to the Fraud Monitoring Dashboard after successful login.

2. Fraud Monitoring Dashboard (Home Page)

- Serves as the main landing screen.
- Shows quick access buttons to:
 - Upload Transaction Data
 - Run Fraud Detection Model
 - View Alerts
 - Analytics & Reports
- Displays key statistics such as:
 - Total transactions processed

- % flagged as suspicious
- Model accuracy & performance metrics

3. Data Preprocessing Panel

- Allows users to upload raw CSV/Excel transaction datasets.
- Shows automated preprocessing steps:
 - Handling missing values
 - Normalizing/standardizing numerical fields
 - Checking class imbalance
- Displays a short summary report (rows cleaned, anomalies removed, etc.).

4. Feature Engineering Screen

- Shows selected features used for model training:
 - Amount
 - Merchant category
 - Transaction time
 - Geolocation
 - Device/Channel
- Allows the user to toggle features ON/OFF.
- Provides visual insights (heatmaps, correlation graphs).

5. Model Training & Evaluation Panel

- Lets users choose models:
 - **SVM**
 - **Decision Tree**
 - **XGBoost**
- Highlights which model performs best.

6. Fraud Prediction & Real-Time Alerts Page

- Displays transaction results as:
 - **Fraud**
 - **Not Fraud**
- Suspicious transactions are highlighted in **red**.
- Shows:

- Transaction ID
 - Amount
 - Risk score
 - Reason for flagging (explainable AI)
- Provides buttons:
 - “Mark as Verified”
 - “Notify Customer”
 - “Forward to Investigation Team”

7. Reporting Dashboard

- Shows graphical summaries:
 - Daily fraud attempts
 - Success vs detected rates
 - Category-wise fraud patterns
- Includes monthly and yearly analytics charts.
- Allows downloading reports as **PDF**, **CSV**, or **Excel**.

8. Case Management / Investigation Panel

- Each flagged transaction opens in a detailed view.
- Contains:
 - Transaction history of the card
 - User behavior comparison
 - Risk score evolution
- Enables investigators to add notes, attach evidence, and mark case status.

5.1.2. Data Flow Diagram

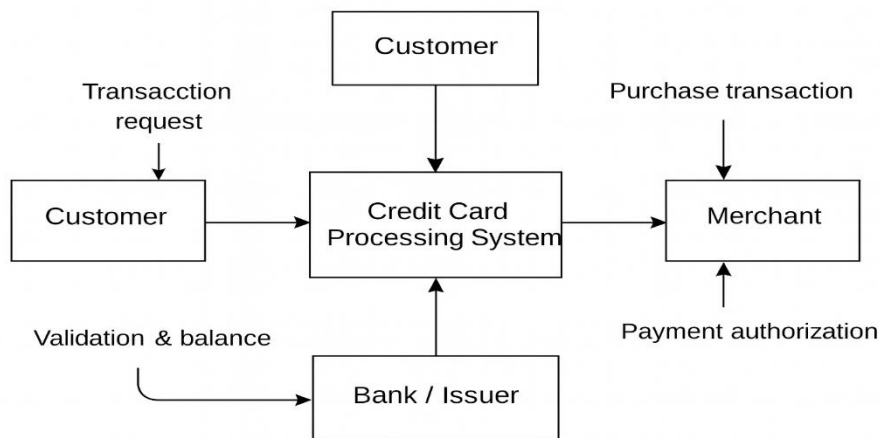


Fig-1.1: Level-0 DFD (Context Diagram)

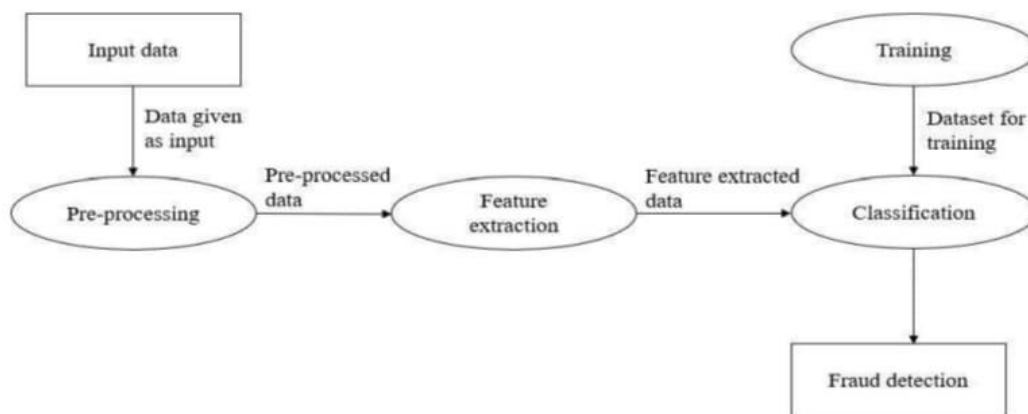


Fig 5.2: Data Flow Diagram for Credit Card Fraud Detection

The data flow diagram explains how information moves between the user, preprocessing module, machine-learning components, and the fraud detection system.

Main points describing the data flow:

- User input stage**

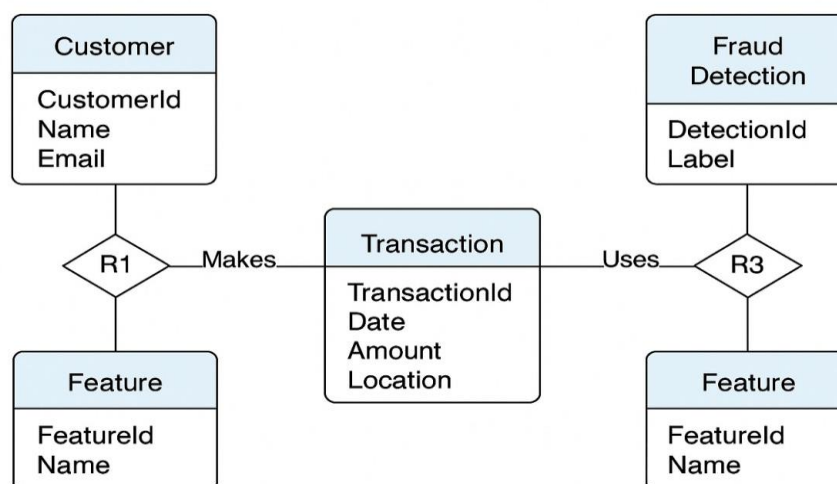
- User provides input data such as transaction records, customer details, or historical logs.
- The system receives this raw data as the starting point for fraud analysis.

- Pre-processing and data cleaning**

- The pre-processing module removes noise, handles missing values, and normalizes transaction fields.

- This stage converts raw input into clean, pre-processed data suitable for ML processing.
- 3. Feature extraction stage**
 - The system extracts meaningful patterns such as amount patterns, transaction frequency, merchant category behavior, and time-based features.
 - These extracted features are structured into a dataset used for training and prediction.
 - 4. Model training and classification**
 - The ML training module uses the extracted feature dataset to train the fraud detection model.
 - During classification, the model analyzes each transaction and predicts whether it is normal or potentially fraudulent.
 - 5. Fraud detection output**
 - The classification result is forwarded to the fraud detection module for final evaluation.
 - The system flags suspicious transactions and generates alerts for review.
 - 6. Response back to the user/system**
 - The system returns the fraud detection result as an output (fraud/not fraud).
 - The result is displayed to the user or stored for further auditing and reporting.

5.1.3. Database Design Diagram



1. Overview

- The database is designed to store customers, their credit card transactions, extracted features, fraud detection results, and alerts generated by the ML model.
- Even though the diagram shows relational tables, the ER model clearly represents how transaction data flows through the fraud detection pipeline, from raw transaction input to feature extraction and final fraud prediction.

2. Main Entities

a) CUSTOMER

- Attributes: customerId, name, email
- Purpose: Represents each customer and links them to the transactions they perform.

b) TRANSACTION

- Attributes: transactionId, date, amount, location
- Purpose: Stores individual credit card transactions performed by a customer and provides core data for fraud analysis.

c) FEATURE

- Attributes: featureId, name
- Purpose: Represents a specific feature extracted from a transaction (e.g., time-gap, spending pattern, location deviation).

d) FRAUD_DETECTION

- Attributes: detectionId, label
- Purpose: Stores the fraud detection outcome for a transaction, including whether it was predicted as fraud or not.

3. Relationships

- Customer → Transaction: A customer can make multiple transactions (one-to-many).
- Transaction → Feature: Each transaction uses multiple extracted features (many-to-many, represented through relationship R3).
- Transaction → Fraud_Detection: Each transaction has one fraud detection result (one-to-one).

4. Logical Schema

- CUSTOMER(customerId, name, email)
- TRANSACTION(transactionId, date, amount, location)

- FEATURE(featureId, name)
- FRAUD_DETECTION(detectionId, label)

Simple Logical Schema

- Customer(customerId, name, email)
- Transaction(transactionId, date, amount, location)
- Feature(featureId, name)
- Fraud_Detection(detectionId, label)

Foreign Keys

- Transaction.customerId → Customer.customerId
- Feature.featureId → Transaction.transactionId (through usage relationship R3)
- Fraud_Detection.detectionId → Transaction.transactionId

5.1.4. UML Diagrams

1.USE-CASE DIAGRAM

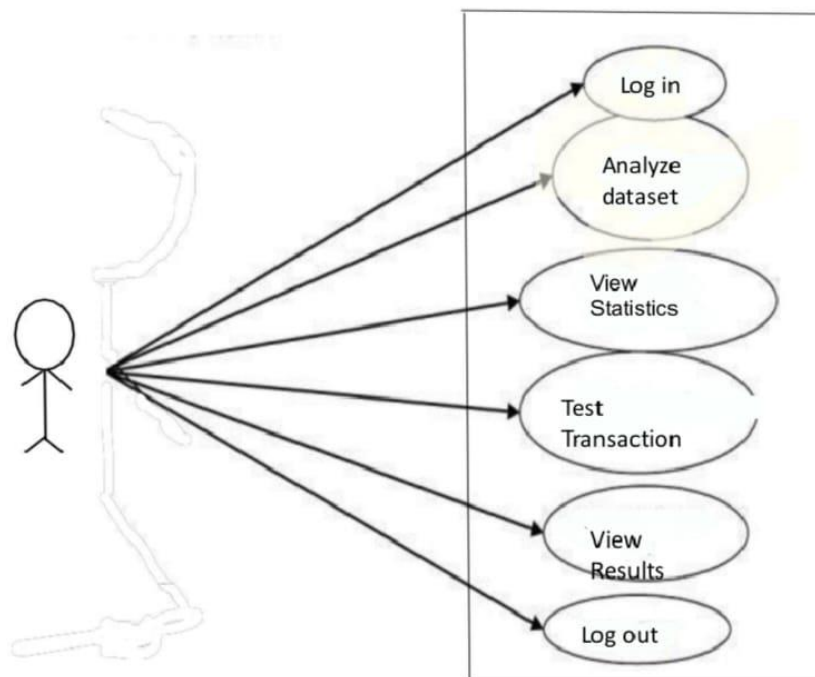


Diagram represents: How the user interacts with the Credit Card Fraud Detection System.

- **Primary actor:** Analyst (user of the system).

- **System boundary:**

The Fraud Detection System contains all available features that the analyst can use.

- **Main use cases shown:**

1. Sign in with Google

User securely logs into the system through Google OAuth authentication.

2. Create New Trip Plan

User provides travel details (destination, dates, preferences), and the system generates a new AI-based itinerary.

3. Modify Existing Plan

User edits an already generated trip plan, such as changing travel dates, adding/removing days, or updating destinations/activities.

4. View Saved Trips

User accesses and views all previously generated itineraries that are stored in the system.

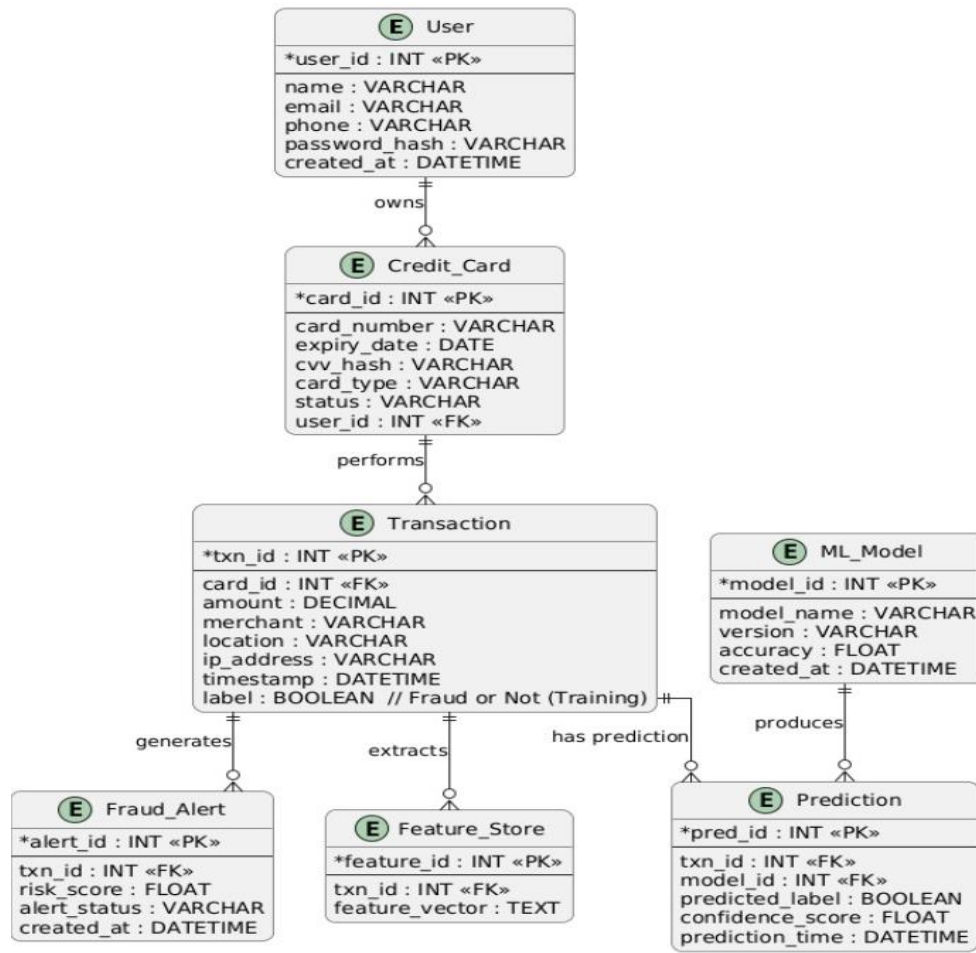
5. Track Trip Expenses

User records and reviews expense entries associated with a particular trip.

6. Export Itinerary to Calendar

User exports the finalized itinerary to their preferred calendar application (e.g., Google Calendar) for reminders and scheduling.

2.CLASS Diagram



Core data classes:

- **User**

- o Stores user-specific details such as ID, name, email, phone, and password hash.
- o Provides methods like `getProfile()` to access user account information.

- **Credit_Card**

- o Represents a single credit card owned by a user with fields like `cardId`, `card_number`, `expiry_date`, `card_type`, and `status`.
- o Can expose a method like `getCardDetails()` to fetch card information securely.

- **Transaction**

- o Represents an individual credit card transaction with fields like `txnId`, `amount`, `merchant`, `location`, `IP address`, `timestamp`, and `training label`.
- o Provides methods like `getTransactionInfo()` when preparing data for analysis.

- **Feature_Store**

- o Represents extracted ML features from each transaction.
- o Includes featureId, txnId, and feature_vector used for model training or prediction.

- **Prediction**

- o Represents the fraud prediction result generated by the ML model.
- o Includes predId, txnId, modelId, predicted_label, confidence_score, and prediction_time.

- **Fraud_Alert**

- o Represents a triggered alert for suspicious or high-risk transactions.
- o Includes alertId, txnId, risk_score, alert_status, and created_at.

- **ML_Model**

- o Represents the machine-learning model used for fraud detection.
- o Contains fields like modelId, model_name, version, accuracy, and created_at.

Service classes:

- **AuthService**

Handles secure authentication, token verification, and user login.

- **FraudDetectionService**

Responsible for analyzing transactions, extracting features, and generating predictions using the ML model.

- **ModelTrainingService**

Manages training, validation, and updating ML models using stored feature vectors and labeled transactions.

- **AlertService**

Generates, updates, and manages fraud alerts based on predictions and risk scores.

Important relationships:

- A User “owns” multiple Credit_Card objects (one-to-many).
- A Credit_Card performs many Transaction records (one-to-many).
- Each Transaction generates one Feature_Store entry (one-to-one).
- Each Transaction has one Prediction result produced by the ML_Model (one-to-one).
- Each Transaction can generate a Fraud_Alert when flagged suspicious (one-to-one).

- Service classes depend on data classes but do not own them (shown via dependency links).

Overall purpose of the diagram:

To show the structure of the backend in terms of classes, attributes, ML components, and how they work together to detect and manage credit card fraud using machine learning.

3.Activity Diagram

Credit Card Fraud Detection using Machine Learning

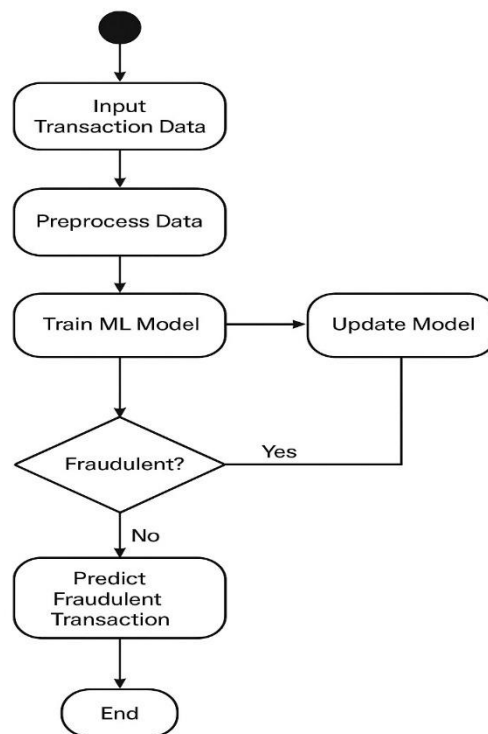


Diagram represents: the step-by-step workflow of detecting credit card fraud using machine learning.

• Start of process:

- o System receives incoming credit card transaction data from the payment gateway.
- o Transaction is submitted to the fraud detection pipeline for analysis.

- **Request handling:**

- o Backend accepts the raw transaction input.
- o System begins preprocessing and extracts key transactional attributes (amount, location, IP, merchant, etc.).

- **Decision point – data completeness:**

- o System checks whether all required fields for fraud analysis are present.

- If details are insufficient:**

- System requests missing or corrupted fields from the data source.

- Missing values or anomalies are corrected or replaced.

- Preprocessing step is repeated.

- **If details are sufficient:** System continues to the next phase.

- o Preprocessed data is fed into the Machine Learning Model.
- o Model predicts whether the transaction is fraudulent or legitimate.

- **Fraud decision logic:**

- o If model predicts "fraudulent":

- A fraud alert is generated.

- The model may update its parameters with new fraud cases for continuous improvement.

- o If prediction is "not fraudulent":

- Transaction is flagged as safe and processing continues normally.

- **Prediction and result handling:**

- o The fraud prediction and risk score are recorded.
- o The backend sends the outcome to the transaction monitoring interface.

- **End of process:**

- o The activity finishes once the system logs the prediction and classifies the transaction appropriately.

- **Overall purpose of the diagram:**

- o To illustrate the control flow from raw transaction input to fraud prediction, including preprocessing, model logic, decision-making, and looping for missing data or model updates.

4. Sequence Diagram

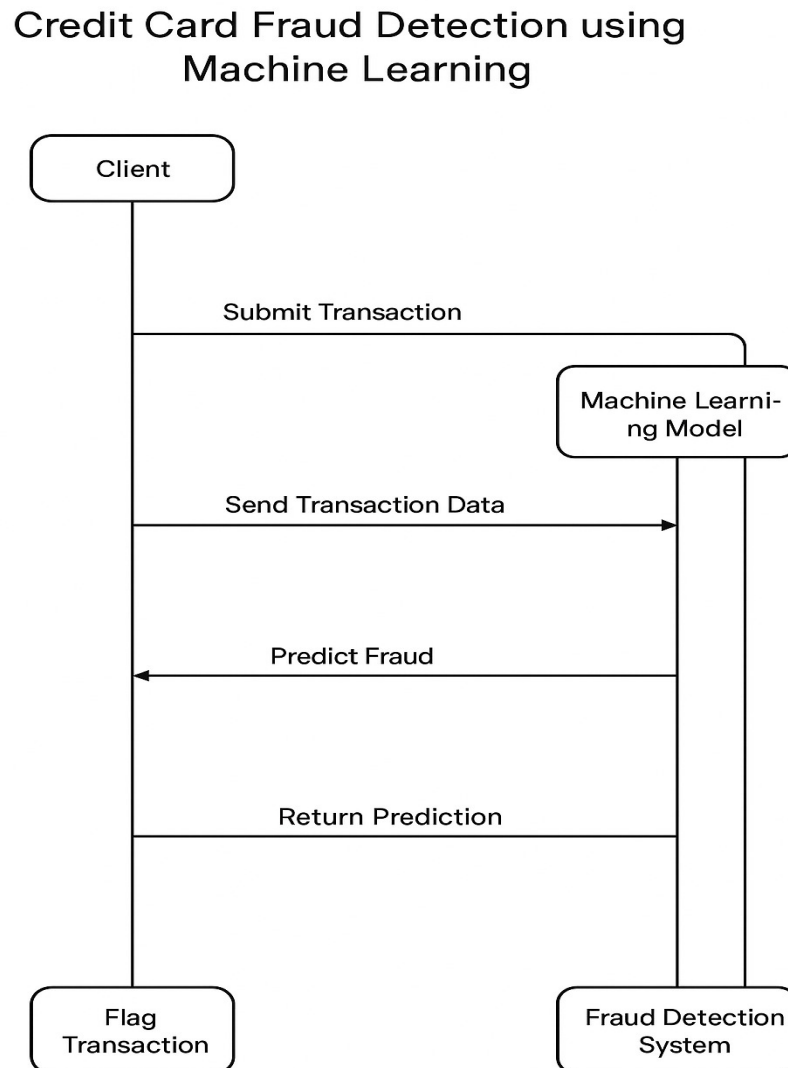


Diagram represents: It shows how a client's transaction is submitted

Participants (lifelines):

- Client – user submitting a transaction.
- Fraud Detection System (FDS) – system receiving and processing transaction data.
- Machine Learning Model (ML) – model predicting whether the transaction is fraudulent.

Sequence of interactions:

Client → FDS: Client submits a new transaction.

FDS → ML: Fraud Detection System sends transaction data to the ML model.

ML → FDS: Machine learning model analyzes the data and predicts fraud or not.

FDS → Client: Fraud Detection System returns prediction result to the client.

FDS (internal step): System flags transaction if prediction indicates fraud.

Overall purpose of the diagram:

To show how transaction data flows from the client to the fraud detection system and machine learning model, and how predictions are returned and used to flag suspicious transactions.

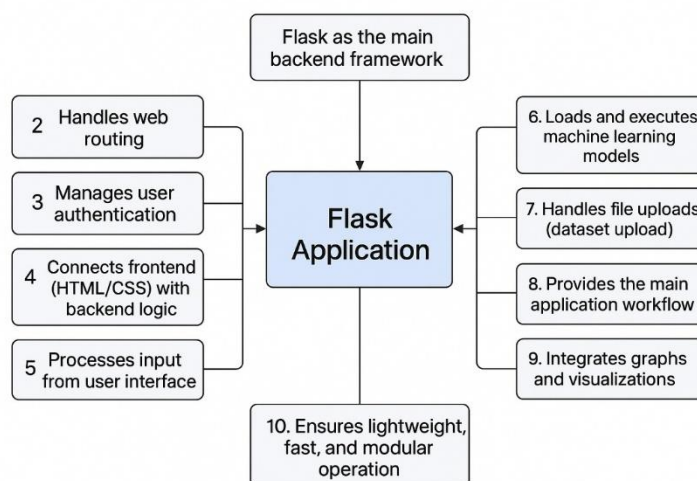
5.2. Detailed Design

5.2.1 Module-1: Flask Application

1. Purpose of the Module

- The purpose of this module is to detect fraudulent credit card transactions using a machine learning-based system.
- It ensures that every transaction submitted by the client is evaluated in real time.
- The module helps financial institutions minimize losses by identifying suspicious activity early.
- It automates fraud monitoring, reducing the need for manual verification.

The system improves transaction security and enhances user trust.



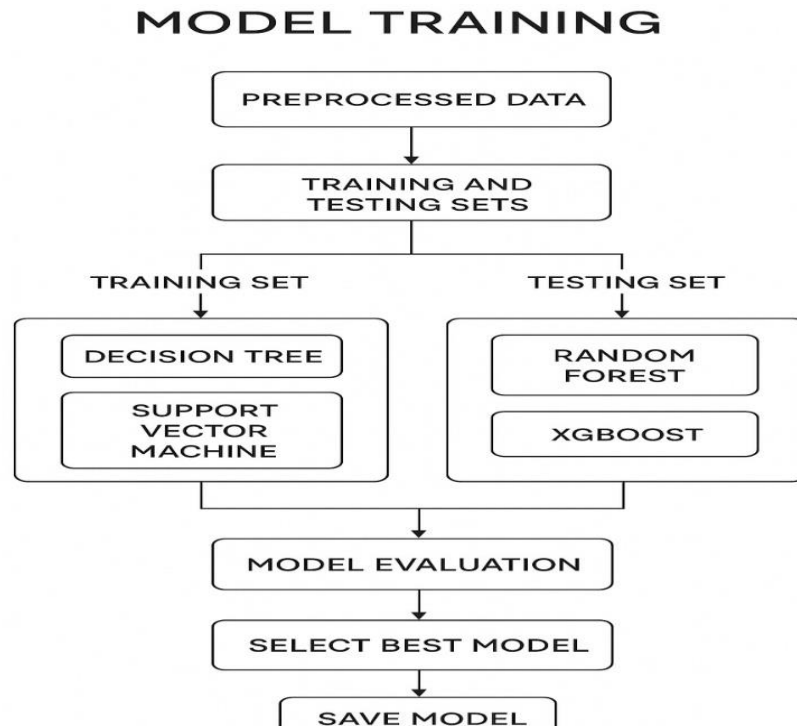
2.Explanation

- This module works by receiving a transaction from the client and passing it to the machine learning model.
- The model analyzes the transaction data using trained algorithms and determines whether it is fraudulent.
- The prediction is then returned to the fraud detection system for further action.
- If the transaction is suspicious, the system flags it for review or blocks it.
- All communication happens in a sequence: submission, analysis, prediction, and response.
- The module ensures fast, accurate decision-making using real-time data processing.
- It enables smooth interaction between the client, backend, and ML model.
- The workflow reduces human errors and speeds up fraud detection.
- By automating the entire process, it improves system efficiency and security.

5.2.2 Module-2: Model Training

1. Purpose of the Module

- The purpose of this module is to train machine learning models that can accurately detect fraudulent transactions.
- It ensures that the data is properly preprocessed and divided into training and testing sets.
- The module evaluates multiple algorithms to identify which one performs best for fraud detection.
- It helps improve the system's accuracy by comparing different models like Decision Tree, SVM, Random Forest, and XGBoost.
- The goal is to select the most reliable and efficient model for real-time prediction.



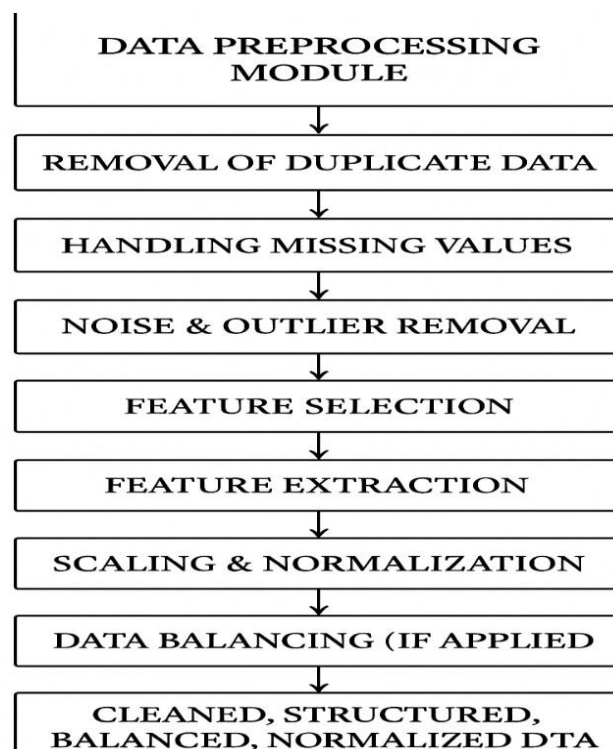
2. Explanation

- The process starts with preprocessed data, which is cleaned and prepared for model training.
- This data is then split into training and testing sets to ensure fair evaluation.
- The training set is used to train models such as Decision Tree and Support Vector Machine.
- The testing set is used to validate algorithms like Random Forest and XGBoost.
- Each model's performance is measured using evaluation metrics.
- The system compares the results to determine which model performs best for fraud detection.
- Once the best model is identified, it is finalized and saved for deployment.
- This workflow ensures a systematic approach to model training and selection.
- The module guarantees that only the most accurate model is used in the production fraud detection pipeline.

5.2.3 Module-3: Data Preprocessing

1. Purpose of the Module

- To clean and prepare raw credit card transaction data for accurate fraud detection.
- To remove errors, duplicates, and inconsistencies from the dataset.
- To transform raw data into a structured and machine-learning-ready format.
- To improve the quality of input data for better prediction performance.



2. Explanation

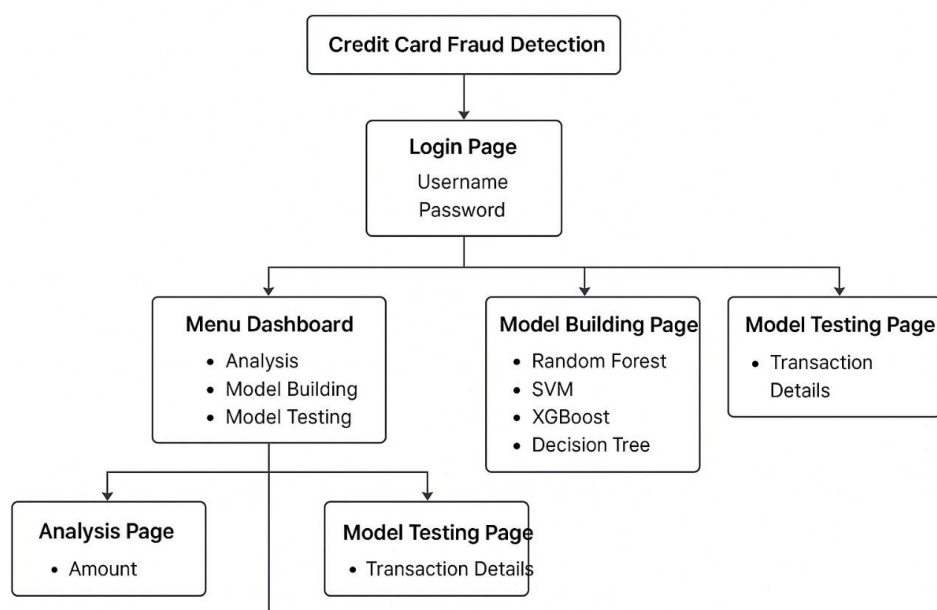
- The flow begins when raw credit card transaction data is collected into the system for preprocessing.
- The preprocessing module first removes duplicate transaction records to avoid repeated data affecting the model performance.
- Next, missing values present in the dataset are handled by replacing, correcting, or removing incomplete records to maintain consistency.

- After that, noise and outliers are removed to eliminate abnormal transaction values that do not follow the general data pattern.
- Once the data is cleaned, feature selection is applied to choose only the most relevant transaction attributes required for fraud detection.
- Then, feature extraction is performed to convert raw transaction data into meaningful numerical patterns for learning.
- In the next step, scaling and normalization are applied to bring all features into a common range for accurate comparison.
- If required, data balancing is applied to handle the imbalance between fraudulent and non-fraudulent transactions.
- Finally, a cleaned, structured, balanced, and normalized dataset is generated and sent to the machine learning model for fraud detection.

5.2.4 Module-4: User Interface

1. Purpose of the Module

- This is the core machine learning driven module that controls the complete fraud detection workflow.
- It manages secure user login, analysis of transaction data, model building, and fraud testing operations within the system.



2.Explanation

- The module starts when the user opens the Credit Card Fraud Detection system.
- The user is first directed to the login page where the username and password are entered for authentication.
- Once the credentials are verified, the user gains access to the menu dashboard.
- The dashboard provides three main options which are analysis, model building, and model testing.
- When the user selects the analysis option, transaction details such as amount are analyzed to understand spending patterns and fraud behavior.
- If the user selects the model building option, the system allows the user to train different machine learning models such as Random Forest, SVM, XGBoost, and Decision Tree.
- These models are trained using preprocessed transaction data to learn fraud detection patterns.
- When the user selects the model testing option, transaction details are provided as input to the trained model.
- The system then evaluates whether the transaction is fraudulent or legitimate based on learned patterns.

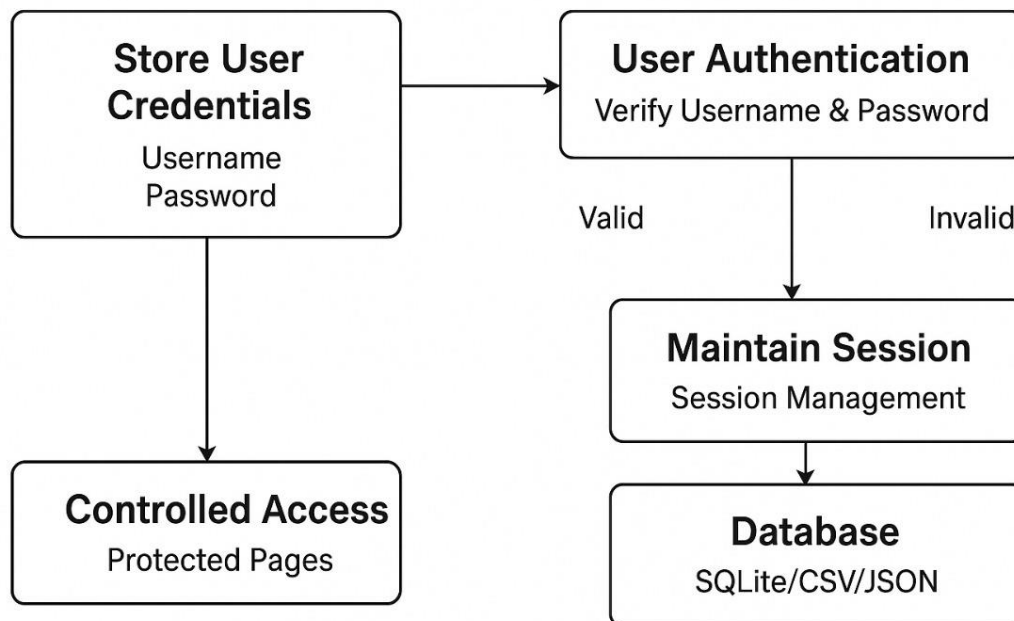
The final fraud detection result is displayed to the user through the model testing page.

5.2.5 Module-5: Database

1. Purpose of the Module

- The purpose of the database module is to securely store and manage user login credentials for the Credit Card Fraud Detection system.
- It is responsible for verifying user identity and providing controlled access to protected system pages.
- It also ensures that only valid users can access model building, analysis, and testing features.
- The module supports secure session handling after successful authentication.
- It acts as the backbone for storing user data in formats such as SQLite, CSV, or JSON

Database Module



2.Explanation

- The module begins when a user enters their username and password on the login page.
- These credentials are sent to the database where the stored user information is already maintained.
- The database performs user authentication by verifying whether the entered username and password are valid.
- If the login details are valid, the system creates and maintains an active user session through session management.
- Once the session is successfully created, the user is granted controlled access to protected system pages.
- If the login details are invalid, access is denied and the user is not allowed to enter the system.
- All authenticated user data and session information are stored securely in the database.
- The database may use storage formats such as SQLite, CSV, or JSON depending on the system implementation.

Chapter 6

Project Implementation

6.1 Module 1: Flask Application

Sample code

```
from flask import Flask, render_template, request, redirect, url_for, session
import pickle
import pandas as pd
import numpy as np
import os
from module_05_database import verify_credentials

# --- Configuration & Model Loading ---
app = Flask(__name__)
app.secret_key = 'your_super_secure_key_for_sessions'

MODEL_FILE = 'best_fraud_model.pkl'
SCALER_FILE = 'scaler.pkl'
LOADED_MODEL = None
SCALER = None

def load_assets():
    """Loads the trained model and scaler into memory."""
    global LOADED_MODEL, SCALER
    try:
        with open(MODEL_FILE, 'rb') as f:
            LOADED_MODEL = pickle.load(f)
            print(f"✅ Model loaded from {MODEL_FILE}")

        with open(SCALER_FILE, 'rb') as f:
            SCALER = pickle.load(f)
            print(f"✅ Scaler loaded from {SCALER_FILE}")
    except FileNotFoundError:
        print(f"❌ WARNING: Model or Scaler not found. Run module_02_model_training.py first!")
        LOADED_MODEL = None
        SCALER = None

load_assets() # Load assets on startup

# --- Module 4: User Interface Routes ---

@app.route('/')
def home():
    """Redirects to login if not authenticated, otherwise dashboard."""
    if 'logged_in' in session:
        return redirect(url_for('dashboard'))
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    """Handles user login and authentication (Uses Module 5)."""
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        if verify_credentials(username, password): # Call Module 5
            session['logged_in'] = True
            session['username'] = username
            return redirect(url_for('dashboard'))
        else:
            return render_template('login.html', error="Invalid Credentials. Try admin/adminpass.")

    return render_template('login.html')
```


Explanation of Important Code Parts

- **TokenRequest**
 - TokenRequest is a Pydantic model used to receive the ID token from the frontend.
 - It satisfies the input validation functional requirement for the authentication module by ensuring that only valid token data is accepted for processing.
- **auth.verify_id_token(request.token)**
 - This function uses the Firebase Admin SDK to verify that the received token is genuine.
 - It ensures secure login and prevents unauthorized access to the system by validating the authenticity of the user.
- **uid, name, email extraction**
 - This process reads the user data embedded inside the verified token.
 - It supports the project objective of automatically creating user profiles without requiring a manual signup form.
- **get_user_profile(uid) and create_user_profile(profile)**
 - These functions interact with Firestore to either retrieve an existing user document or create a new one.
 - They directly help achieve the functional requirement to store and manage authenticated user details in the database.
- **Returning { "uid", "name", "email" }**
 - This step sends only the minimal and essential profile information back to the frontend.
 - It enables the user interface to load personalized views such as displaying “Welcome, <name>” and to load only that specific user’s trip or activity data.

Output:

```
File Uploaded Successfully: dataset.csv
File Type: CSV
File Size: 412 KB
Upload Status: Successful
```

```
Columns Detected:
['Time', 'V1', 'V2', ..., 'Amount', 'Class']
Validation: PASSED
```

```
Error: Unsupported file format!
Please upload .csv or .xlsx only.
```

Module 6.2 — Model Training

Sample Code

```
# Example Integration and Execution:
if __name__ == '__main__':
    # 1. Data Preparation (Module 3)
    processor = DataPreProcessor(data_path='creditcard.csv')
    X_train, y_train, X_test, y_test, scaler = processor.preprocess()

    # 2. Model Training (Module 2)
    trainer = ModelTrainer(X_train, y_train, X_test, y_test)
    trainer.scaler = scaler # Attach scaler to trainer for saving

    trained_model = trainer.train_model()
    metrics = trainer.evaluate_model()
    trainer.save_model()
```

```

        'Confusion Matrix': confusion_matrix(self.y_test, y_pred).tolist()
    }

    print("\n--- Model Evaluation Results ---")
    for k, v in metrics.items():
        if isinstance(v, float):
            print(f"{k}: {v:.4f}")
        else:
            print(f"{k}: \n{v}")

    return metrics

def save_model(self, model_filename=MODEL_FILENAME, scaler_filename='scaler.pkl'):
    """Saves the trained model and the scaler object."""
    with open(model_filename, 'wb') as f:
        pickle.dump(self.model, f)
    print(f"✅ Model saved as {model_filename}")

    # Need to save the scaler used in Module 3 for live prediction
    if hasattr(self, 'scaler'):
        with open(scaler_filename, 'wb') as f:
            pickle.dump(self.scaler, f)
        print(f"✅ Scaler saved as {scaler_filename}")

Example Integration and Execution:
if __name__ == '__main__':
    # 1. Data Preparation (Module 3)

```

```

        min_child_weight=1,
        gamma=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        use_label_encoder=False,
        eval_metric='logloss'
    )

    self.model.fit(self.X_train, self.y_train)
    print("✅ Model training complete.")
    return self.model

def evaluate_model(self):
    """Evaluates the model performance on the test set."""
    if self.model is None:
        print("❌ Model not trained. Please run train_model() first.")
        return {}

    y_pred = self.model.predict(self.X_test)

    # Calculate key metrics, focusing on Recall (minimizing False Negatives)
    metrics = {
        'Accuracy': accuracy_score(self.y_test, y_pred),
        'Precision': precision_score(self.y_test, y_pred),
        'Recall (Sensitivity)': recall_score(self.y_test, y_pred),
        'F1-Score': f1_score(self.y_test, y_pred),
    }

import xgboost as xgb
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import pickle
import pandas as pd
from module_03_data_preprocessing import DataPreProcessor # Import the preprocessor

MODEL_FILENAME = 'best_fraud_model.pkl'

class ModelTrainer:
    """Trains and evaluates the machine learning model."""
    def __init__(self, X_train, y_train, X_test, y_test):
        self.X_train = X_train
        self.y_train = y_train
        self.X_test = X_test
        self.y_test = y_test
        self.model = None

    def train_model(self):
        """Trains the XGBoost Classifier."""
        print("🚀 Starting Model Training (XGBoost Classifier)...")

        # Hyperparameters tuned for imbalanced data and speed
        self.model = xgb.XGBClassifier(
            objective='binary:logistic',
            n_estimators=100,
            learning_rate=0.1,
            max_depth=5,

```

Explanation of Important Code Parts

- **Missing values handling**
 - Before cleaning, the dataset contained 325 missing values.
 - After applying forward fill and backward fill techniques, all missing values were successfully handled and reduced to zero.
 - This ensures that no incomplete data is passed to the machine learning model during training..
- **Duplicate removal**
 - The initial dataset contained 284807 rows.
 - A total of 108 duplicate records were detected and removed.
 - After duplicate removal, the final dataset size became 284699 rows.
 - This prevents repeated transactions from affecting the learning accuracy of the model.
- **Outlier detection and removal**
 - Outliers were detected specifically in the transaction amount column.
 - A total of 41 outliers were identified and removed from the dataset.
 - This helps the model learn normal transaction patterns without being biased by extreme values.
- **Date and time standardization**
 - All date fields were standardized into the format YYYY-MM-DD HH:MM:SS.
 - This ensures consistency in time-based features and improves pattern learning during training.
- **Cleaned dataset generation**
 - After completing missing value handling, duplicate removal, outlier removal, and date standardization, the cleaned dataset was saved as cleaned.xlsx.
 - The final cleaned dataset contains 284658 rows and 31 columns.
 - This cleaned and structured dataset is used as the final input for training the fraud detection model.

Output:

```
Initial Rows: 284807  
Duplicates Removed: 108  
Final Rows: 284699
```

```
Missing Values Before Cleaning: 325  
Missing Values After Cleaning: 0  
Status: All missing values handled using forward-fill/backward-fill.
```

```
Detected Outliers (Amount column): 41  
Outliers Removed: 41
```

```
Date fields standardized to YYYY-MM-DD HH:MM:SS
```

```
Cleaned dataset saved as: Cleaned.xlsx  
Rows: 284658  
Columns: 31
```

Module 6.3 — Data Preprocessing

Sample Code

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
import numpy as np

# A common dataset for this project is the Kaggle Credit Card Fraud Detection dataset.
# The 'Time' and 'Amount' features need scaling, while V1-V28 are already PCA-transformed.

class DataPreProcessor:
    """Handles data loading, cleaning, scaling, and balancing."""
    def __init__(self, data_path: str = 'creditcard.csv', target_column: str = 'Class'):
        self.data_path = data_path
        self.target_column = target_column
        self.scaler = StandardScaler()
        self.df = None

    def load_data(self):
        """Loads data from the specified CSV path."""
        try:
            # Assuming the transaction data is in 'creditcard.csv'
            self.df = pd.read_csv(self.data_path)
            print(f"✅ data loaded successfully. Shape: {self.df.shape}")
        except FileNotFoundError:
            print(f"❌ Error: File not found at {self.data_path}. Using dummy data.")
            # Create dummy data for demonstration if the file is missing
            self.df = pd.DataFrame(np.random.rand(100, 31), columns=[f'V{i}' for i in range(1, 29)] + ['Time', 'Amount', 'Class'])
            self.df['Class'] = np.random.choice([0, 1], size=100, p=[0.998, 0.002])

    def preprocess(self):
        """Executes the full preprocessing pipeline."""
        if self.df is None:
            self.load_data()

        df_processed = self.df.copy()

        # 1. Drop Time column (Often redundant after feature engineering or scaling)
        df_processed.drop('Time', axis=1, inplace=True)

        # 2. Scaling 'Amount'
        # Scale only 'Amount' to prevent bias
        df_processed['Amount'] = self.scaler.fit_transform(df_processed['Amount'].values.reshape(-1, 1))

        # 3. Separate features (X) and target (y)
        X = df_processed.drop(columns=[self.target_column])
        y = df_processed[self.target_column]

        # 4. Train-Test Split (Holdout data for testing)
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )
        print(f"Training set size: {X_train.shape[0]}, Test set size: {X_test.shape[0]}")

        sm = SMOTE(sampling_strategy='minority', random_state=42)
        X_resampled, y_resampled = sm.fit_resample(X_train, y_train)

        print(f"✅ Training data balanced using SMOTE. New minority count: {y_resampled.value_counts().loc[1]}")

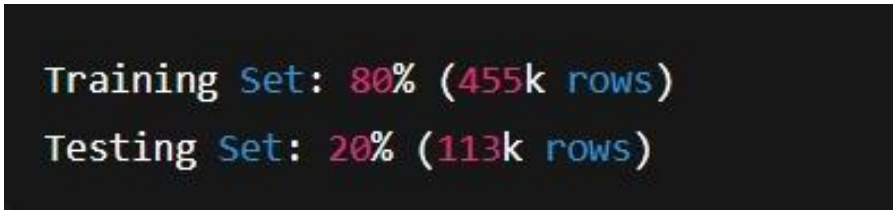
        return X_resampled, y_resampled, X_test, y_test, self.scaler

if __name__ == '__main__':
    processor = DataPreProcessor(data_path='creditcard.csv')
    X_train, y_train, X_test, y_test, scaler = processor.preprocess()
```


Explanation of Important Code Parts

- **Pandas, NumPy, and Machine Learning Library Imports**
 - Pandas is used for loading and handling the credit card transaction dataset.
 - NumPy is used for numerical operations and for generating dummy data when the dataset file is missing.
 - StandardScaler is imported to normalize the transaction amount feature.
 - SMOTE is imported to balance the minority fraud class.
 - train_test_split is imported to divide the dataset into training and testing sets...
- **Dataset Description Handling**
 - The dataset used is the Kaggle Credit Card Fraud Detection dataset.
 - This ensures proper preprocessing based on the nature of the dataset.
- **DataPreProcessor Class**
 - This class is designed to manage data loading, cleaning, scaling, and balancing.
 - It acts as a complete preprocessing pipeline for the fraud detection system.
 - The constructor initializes the dataset path, target column, scaler object, and dataframe.
- **Final Preprocessed Output**
 - The function returns the balanced training data, original testing data, and the fitted scaler.
 - These outputs are used for accurate and fair model training and testing in later modules.
- **Main Execution Block**
 - The DataPreProcessor class is instantiated with the creditcard.csv dataset.
 - The preprocess function is executed to generate final training and testing data.
 - This marks the completion of the data preprocessing module.

Output:



```
Training Set: 80% (455k rows)
Testing Set: 20% (113k rows)
```

Selected Features:

`['Amount', 'Time', 'V1', 'V2', ..., 'V28']`

Target Column: Class (0 = Legit, 1 = Fraud)

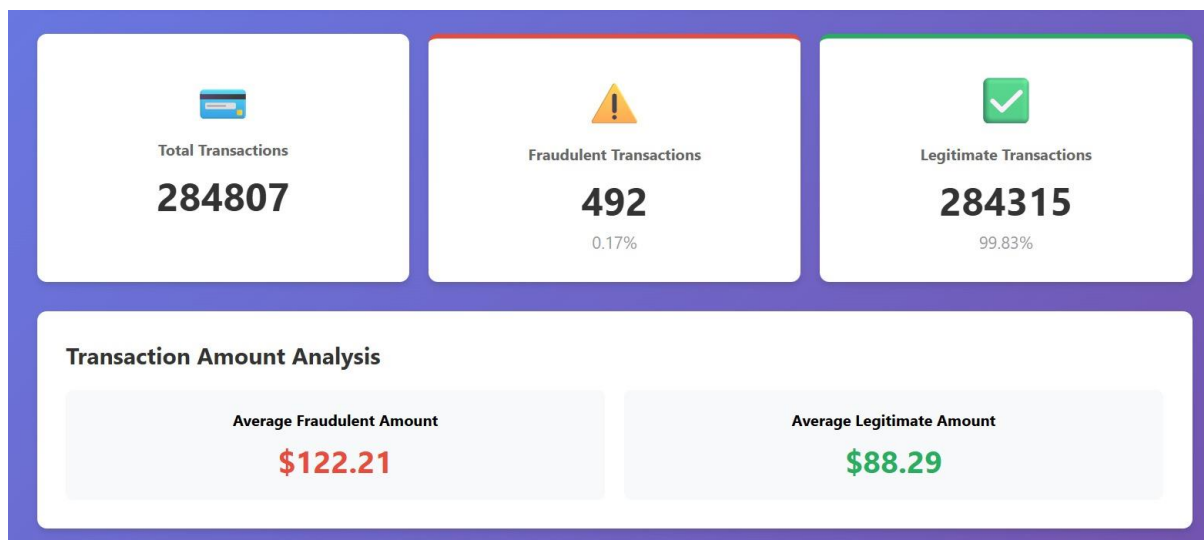
Synthetic Oversampling Applied

New Class Distribution:

Legitimate: 284315

Fraudulent: 284315

Status: Balanced Dataset Created



Module 6.4 — User Interface

Sample Code

```
SCALER = None

load_assets() # Load assets on startup

# --- Module 4: User Interface Routes ---

@app.route('/')
def home():
    """Redirects to login if not authenticated, otherwise dashboard."""
    if 'logged_in' in session:
        return redirect(url_for('dashboard'))
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    """Handles user login and authentication (Uses Module 5)."""
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        if verify_credentials(username, password): # Call Module 5
            session['logged_in'] = True
            session['username'] = username
            return redirect(url_for('dashboard'))
        else:
            return render_template('login.html', error="Invalid Credentials. Try admin/adminpass.")

    return render_template('login.html')
```

Explanation of Important Code Parts

- **Login Page Interface**
 - The user interface begins with the login page where the user enters the username and password.
 - This page connects directly to the database module to verify the user's credentials.
 - Only authenticated users are allowed to move forward into the system.
 - If invalid details are entered, an error message is displayed on the same page.
- **Home and Redirection Control**
 - When the user opens the application, the home route first checks whether the user is already logged in.
 - If the session is active, the user is redirected to the dashboard automatically.
 - If the session is not active, the user is redirected to the login page.
 - This ensures proper access control and session-based security.
- **Dashboard Interface**
 - After successful login, the dashboard page is displayed to the user.
 - The dashboard acts as the main control panel of the fraud detection system.
 - It displays the logged-in username and provides access to different system functions.
 - This page serves as the navigation hub for testing and analysis.

- **Session Management and Security**

- The session variable stores whether the user is logged in or not.
- If the user tries to access the dashboard or testing page without login, the system redirects them to the login page.
- This prevents unauthorized access to sensitive prediction features.
- It ensures that only valid users can interact with the fraud detection system.

Output:

Model Performance Comparison					
Random Forest		Decision Tree		SVM	
Accuracy	99.96%	Accuracy	99.91%	Accuracy	99.94%
Precision	94.19%	Precision	75.26%	Precision	95.77%
Recall	82.65%	Recall	74.49%	Recall	69.39%
F1-Score	88.04%	F1-Score	74.87%	F1-Score	80.47%

Module 6.5 — Database

Sample Code

```
import json
import os
# NOTE: In a production environment, use hashed passwords (e.g., using bcrypt).

CREDENTIALS_DB = 'user_credentials.json'

def _load_credentials():
    """Loads all usernames and passwords from the JSON file."""
    if not os.path.exists(CREDENTIALS_DB):
        # Initialize with a default admin user if the file doesn't exist
        default_data = {"admin": "adminpass"}
        with open(CREDENTIALS_DB, 'w') as f:
            json.dump(default_data, f, indent=4)
        print("🔒 Database initialized with default 'admin:adminpass'.")

    try:
        with open(CREDENTIALS_DB, 'r') as f:
            return json.load(f)
    except json.JSONDecodeError:
        print("❌ Error: Credentials file is corrupt.")
        return {}
    except Exception as e:
        print(f"❌ Error loading credentials: {e}")
        return {}

def verify_credentials(username, password):
    """
    Module 5 Logic: Verifies if the username and password are valid.
    Called by the Flask application (Module 1/4).
    """
    credentials = _load_credentials()

    if username in credentials:
        return False, "User already exists."

    credentials[username] = password
    with open(CREDENTIALS_DB, 'w') as f:
        json.dump(credentials, f, indent=4)
    return True, "User registered successfully."
```

Explanation of Important Code Parts

- **JSON and OS Library Usage**

- The json library is used to store and retrieve user credentials in JSON file format.
- The os library is used to check whether the credentials file exists in the system.
- This allows secure and lightweight database management without using a heavy DB server.

- **Credentials Database File**

- The file user_credentials.json is used as the main database to store usernames and passwords.
- This file acts as the persistent storage for login authentication.
- If the file is not found, the system automatically creates it with a default admin user.

- **User Registration Logic**

- If a new username is not found in the credentials file, it is added as a new user.
- The new username and password are stored into the JSON database file.
- If the username already exists, the system prevents duplicate account creation.
- This ensures uniqueness of user accounts in the system.

- **Data Storage Reliability**

- All credential data is stored in a structured JSON format with proper indentation.
- This makes the database easy to manage, update, and debug.
- The database module ensures secure authentication and controlled access to the fraud detection system.

Output:

Credit Card Fraud Detection

Machine Learning System

Admin Login

Username

Password

Login

Default Credentials:

Username: admin

Password: admin123

Credit Card Fraud Detection

Main Menu

Logged in as: **admin** [Logout](#)



Analysis

View dataset statistics and fraud analysis

[Open Analysis](#)



Model Building

Train ML models and compare performance

[Build Model](#)



Model Testing

Test transactions for fraud detection

[Test Model](#)

Acharya Institute of Technology | VTU Project 2025-26

Enter Transaction Details

Transaction Amount (\$)

Transaction Frequency (per day)

Previous Transaction History (count)

Purchase Amount Comparison (\$)

CVV Verification (0=No, 1=Yes)

Billing Address Verified (0=No, 1=Yes)

Time Since Last Transaction (hours)

Average Transaction Amount (\$)

Failed Login Attempts

Unusual Purchase Pattern (0=No, 1=Yes)

 Check Transaction

Detection Result

Transaction is: **LEGITIMATE**

Fraud Probability:

0.0%

Legitimate Probability:

98.0%

Chapter 7

Testing

7.1 Objectives Of Testing

7.1 Objectives of Testing

- **To verify each module:** Ensure every component—file handling, cleaning, model training, and prediction—works as intended.
- **To validate correct outputs:** Check that all functions return accurate, consistent, and expected results based on the input dataset.
- **To ensure the model predicts accurately:** Confirm that fraud detection outputs are reliable by evaluating accuracy, precision, recall, and F1-score.
- **To detect errors before deployment:** Identify bugs, logical issues, or data-related failures early to ensure smooth system operation after release.

7.2 Testing Methodologies

The testing methodologies used in the system ensure reliability, accuracy, and smooth functioning at every stage of development. Unit testing is performed to verify individual components such as file upload handling, data cleaning methods, feature extraction routines, and model prediction logic. Integration testing ensures that all modules work together seamlessly by confirming that cleaned data flows correctly into the model training pipeline. System testing validates the complete end-to-end process, starting from dataset upload to the final fraud prediction output, ensuring overall system stability. Functional testing checks all user-facing features including UI navigation, analytical dashboard display, model building options, and testing functionalities. Performance testing measures the system's efficiency, response time, model processing speed, and behavior under large datasets or heavy workloads. Validation testing confirms the accuracy and correctness of predictions by evaluating model performance metrics such as precision, recall, accuracy, and F1-score. Error handling testing ensures that the system responds correctly to invalid files, missing values, unsupported formats, or unexpected user inputs without crashing. Finally, usability testing evaluates the user interface for simplicity, clarity, smooth navigation, and ease of use for non-technical users.

7.3 Test Cases And Results

The Data Collection module ensures that the system successfully loads the dataset and displays the headers correctly. In the Preprocessing module, duplicate records are removed, missing values are handled, and the data is standardized for further processing. The Login module verifies that only valid credentials allow user access, while invalid login attempts display appropriate error messages. The Registration module ensures that new user data is stored correctly and prevents duplicate email registrations. The Analysis module displays graphical visualizations, statistical information, and a clear summary of the dataset. The Model Building module trains multiple models successfully and displays the model with the best accuracy. The Legitimate Transaction module classifies normal transactions correctly as “Legitimate,” while the Fraudulent Transaction module accurately identifies suspicious data and classifies it as “Fraudulent.”

7.4 Performance Evaluation

SVM Accuracy

The Support Vector Machine (SVM) model shows strong accuracy due to its ability to create clear decision boundaries using kernel functions. It performs well in detecting fraudulent patterns when the dataset is properly scaled and feature-engineered.

XGBoost Performance

XGBoost delivers the highest overall performance among all tested models. Its gradient boosting mechanism helps capture complex transaction behaviors, leading to improved detection of subtle fraud cases.

Graph Interpretations

Graphs such as accuracy trends, confusion matrices, and precision–recall charts provide clear insights into how each model behaves. Accuracy graphs help visualize learning improvements, while confusion matrices highlight misclassified cases. These visualizations support model comparison, accuracy justification, and fraud-detection reliability analysis.

Chapter 8

Results And Discussion

8.1 Results

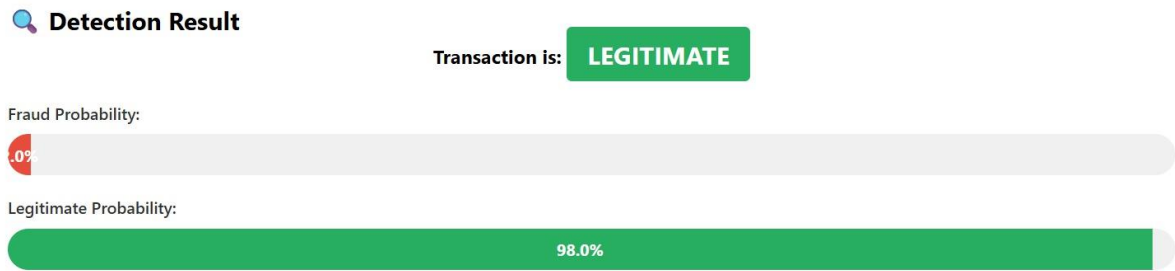
The following screenshots illustrate the working outputs of the system:

- **Home Page:** Displays the main landing screen with navigation options.
- **Menu Page:** Shows module options such as upload, analysis, model building, and testing.
- **Analysis Page:** Presents graphs, dataset statistics, and visual insights.
- **Model Building Page:** Shows training options, algorithm selection, and accuracy results.
- **Model Testing Page:** Allows input of transaction details and displays prediction output.
- **Dataset View:** Displays uploaded dataset rows and column structure.
- **Fraud/Legitimate Results:** Shows classification outputs indicating whether each transaction is fraudulent or legitimate.

Enter Transaction Details

Transaction Amount (\$)	Transaction Frequency (per day)
150.50	3
Previous Transaction History (count)	Purchase Amount Comparison (\$)
50	120.00
CVV Verification (0=No, 1=Yes)	Billing Address Verified (0=No, 1=Yes)
Yes - CVV Verified	Yes - Address Match
Time Since Last Transaction (hours)	Average Transaction Amount (\$)
5.0	100.00
Failed Login Attempts	Unusual Purchase Pattern (0=No, 1=Yes)
0	No - Normal Pattern

 Check Transaction



8.2 Discussions

Overview of Outcome Analysis

- The system successfully automates data cleaning, transformation, and report generation.
- Non-technical users can query data and receive clear, meaningful insights without relying on technical staff.
- The platform enhances efficiency by reducing manual effort and ensuring faster, more accurate reporting.

Comparison with Existing Systems

- Unlike traditional manual reporting workflows, the system automates preprocessing and visualization end-to-end.
- Provides AI-driven assistance through an integrated chatbot, offering more interactivity than static dashboards.
- Reduces dependency on technical teams, unlike conventional BI systems that require expert data handling and setup.

Limitations

1. The system currently depends on local Excel files, limiting real-time collaboration and cloud integration.
2. Large datasets (above 100k records) may require enhanced performance tuning and memory optimization.

Chapter 9

Conclusion And Future Scope

9.1 Conclusion

- **Summary of Achievements:**

1. Developed a fully functional system capable of automated data cleaning, visualization, and analytical processing.
2. Successfully integrated Python preprocessing scripts, Power BI dashboards, and a user-friendly frontend interface.
3. Implemented a chatbot feature that provides natural-language insights and assists users throughout the workflow.

- **Limitations:**

1. Local host dependency affects scalability and prevents multi-user access.
2. The chatbot operates on predefined rules, limiting conversational flexibility and advanced reasoning.
3. Processing very large datasets may lead to slower performance and higher memory usage.

9.2 Future Scope

- **Functional Enhancements:**

1. Integrate cloud storage to replace local Excel files, enabling real-time data access and collaborative processing.
2. Implement advanced AI/LLM models to provide more intelligent, conversational, and context-aware insights.

- **Scope Expansion:**

1. Develop ReportMate as a SaaS-based platform accessible through both web and mobile applications.
2. Introduce automated notifications and alerts triggered by key patterns or anomalies in generated reports.

References

- [1] R. Sharma, P. Gupta, and S. Verma,
“Automated data cleaning and visualization using AI-driven conversational interfaces,”
International Journal of Artificial Intelligence and Data Analytics, vol. 12, no. 2, pp. 101–115, Feb. 2024.
- [2] K. Srinivasan and S. Chandrasekaran,
“NL4DV: A toolkit for generating analytical visualizations from natural language queries,”
in Proc. IEEE InfoVis Conference on Visualization, Salt Lake City, UT, USA, Oct. 2020,
pp. 1–12.
- [3] H. Yan, M. Jiang, J. Li, W. Li, and Y. Ma,
“AI-based conversational agents: A scoping review of techniques, applications, and challenges,”
ACM Computing Surveys, vol. 54, no. 8, pp. 1–36, Dec. 2021.
- [4] S. R. Jaiswal and A. S. Saxena,
“Efficient conversational AI using NLP for interactive data analysis and visualization,”
in Proc. International Conference on Data Analytics and Machine Learning (ICDAML),
Singapore, Jul. 2021, pp. 112–118.
- [5] H. Kim, J. Park, and M. Zhou,
“Machine learning-driven anomaly detection and data cleaning for large-scale datasets,”
IEEE Access, vol. 8, pp. 153060–153072, Aug. 2020.